
DISENTANGLING GRADIENT QUALITY FROM ARCHITECTURE IN RECURSIVE REASONING MODELS

Jatin Jani^{*†}
Independent Researcher
janijatin18@gmail.com

ABSTRACT

Recent work on recursive reasoning has produced two competing architectures: the Hierarchical Reasoning Model (HRM) and Tiny Recursive Models (TRM), which differ dramatically in performance, with 55.0% versus 87.4% exact accuracy on Sudoku-Extreme. However, TRM changed both the architecture (dual-network hierarchy $\rightarrow$ single flat network) *and* the gradient method (1-step approximation $\rightarrow$ full backpropagation through time) simultaneously, making it impossible to attribute performance differences to either factor in isolation. In this work, we disentangle these two variables by running TRM’s flat architecture with HRM’s 1-step gradient approximation. Across an identical training setup (384 hidden dimensions, 2 layers, 10,000 epochs, 1,000 Sudoku examples), we find that exact accuracy collapses from 18.9% (full BPTT) to 2.2% (1-step gradient), an $8.6\times$ reduction. Token-level accuracy drops more modestly from 71.6% to 65.9%. Our results demonstrate that an $8.6\times$ gap produced by varying only the gradient method on identical architecture reveals that TRM’s advantage over HRM stems not solely from architecture, but the gradient method also played a critical role in driving its performance. Future work on recursive reasoning must control for gradient method when comparing architectures.

Keywords Recursive reasoning · gradient methods · architecture design · backpropagation through time · empirical methodology

1 Introduction

Recursive reasoning, the ability to iteratively refine computations through repeated application of a neural network, has emerged as a promising alternative to chain-of-thought prompting for complex puzzle-solving tasks. Two recent models exemplify this paradigm: the Hierarchical Reasoning Model (HRM) [Wang et al.(2025)Wang, Li, Sun, Chen, Liu, Wu, Lu, Song, and Yadkori] and Tiny Recursive Models (TRM) [Jolicoeur-Martineau(2025)]. Both operate on small datasets ($\sim 1,000$ examples) without pretraining, yet achieve remarkably different results. On Sudoku-Extreme, HRM achieves 55.0% exact accuracy while TRM achieves 87.4%, a gap of 32.4 percentage points.

At first glance, this gap appears to validate TRM’s central claim: that “less is more”: that a simple flat architecture outperforms HRM’s complex dual-network hierarchy. However, TRM changed two independent variables simultaneously:

1. **Architecture:** HRM uses two separate networks (an H-module for abstract planning and an L-module for detailed computation, operating at different update frequencies). TRM collapses these into a single shared network.
2. **Gradient method:** HRM uses a 1-step gradient approximation based on the Implicit Function Theorem [Bai et al.(2019)Bai, Kolter, and Koltun], requiring only $O(1)$ memory regardless of recursion depth. TRM uses full backpropagation through time (BPTT), requiring $O(T)$ memory.

^{*}Code: <https://github.com/heyiamjj/disentangling-gradients-recursive-reasoning>

[†]Models: <https://huggingface.co/heyiamjj/disentangling-gradients-recursive-reasoning>

Table 1: The confound in the prior literature. HRM and TRM differ along two dimensions simultaneously, making direct comparison invalid. The community compared HRM (top-left, 55.0%) to TRM (bottom-right, 87.4%) and attributed the gap to architecture, ignoring the gradient confound.

	Dual L/H (HRM)	Single flat (TRM)
1-step gradient ($O(1)$)	HRM: 55.0%	?
Full BPTT ($O(T)$)	?	TRM: 87.4%

Either change could explain the performance improvement, or both. To our knowledge, no prior work has systematically isolated these two variables, and the community has subsequently attributed TRM’s success to its architectural simplification [ARC Prize Foundation(2025)].

In this paper, we resolve the confound through a controlled experiment. We take TRM’s exact flat architecture and apply HRM’s 1-step gradient approximation, keeping all other variables constant (model size, training data, hyperparameters, hardware). This fills the unexplored cell of the resulting 2×2 factorial design (Table 1 and Table 3).

Our key finding is that gradient quality dominates: the 1-step gradient approximation reduces exact accuracy by $8.6\times$ (from 18.9% to 2.2%), while token-level accuracy drops by 8.0% (71.6% to 65.9%). The flat architecture *can* learn, but the crippled gradient signal prevents it from learning coherent, complete solutions.

We make the following contributions:

- We provide the first controlled experiment isolating gradient quality from architecture design in recursive reasoning models.
- We demonstrate that gradient method alone can produce an order of magnitude difference ($8.6\times$ gap, Table 3) in recursive reasoning performance, confounding direct architecture comparisons.
- We establish a clean 2×2 factorial framework for future research on recursive architectures, with the remaining cell (HRM architecture + full BPTT) left as a natural next experiment.

2 Background

2.1 Hierarchical Reasoning Model (HRM)

HRM [Wang et al.(2025)Wang, Li, Sun, Chen, Liu, Wu, Lu, Song, and Yadkori] consists of two recurrent modules: a high-level module (f_H) updated every T timesteps for abstract planning, and a low-level module (f_L) updated every timestep for detailed computation. Over N high-level cycles of T low-level steps each, the model performs $N \times T$ total computational steps.

Critically, HRM uses a 1-step gradient approximation inspired by Deep Equilibrium Models [Bai et al.(2019)Bai, Kolter, and Koltun]. During backpropagation, only the *final* step of each module builds a computation graph; all preceding steps execute under `torch.no_grad()`. This provides $O(1)$ memory scaling, constant with respect to recursion depth, but the gradient signal propagates through only 1 step rather than the full $N \times T$ chain. HRM justifies this approximation via the Implicit Function Theorem, arguing that the recurrent dynamics converge to a fixed point where single-step gradients suffice.

HRM achieves 55.0% exact accuracy on Sudoku-Extreme with 27M parameters trained on $\sim 1,000$ examples. It also reports 40.3% on ARC-AGI-1 and strong results on maze pathfinding.

2.2 Tiny Recursive Models (TRM)

TRM [Jolicoeur-Martineau(2025)] simplifies HRM’s architecture dramatically: instead of two separate networks, TRM uses a single 2-layer network that updates both a latent reasoning feature z and a predicted answer y . The model recursively improves its answer: n latent reasoning steps update z conditioned on the input x , current answer y , and current latent z ; then one step updates y given y and z . This process repeats for T supervision segments, with hidden states detached between segments to prevent gradient flow across them.

Crucially, within each supervision segment, TRM uses *full* BPTT through all n latent steps and the y update. This means gradients flow through 7 differentiable transformer passes per segment (with $n = 6$ and a final y update), compared to HRM’s single differentiable pass.

TRM achieves 87.4% exact accuracy on Sudoku-Extreme with only 7M parameters. An independent analysis [ARC Prize Foundation(2025)] found that deep supervision doubled accuracy (19% $\rightarrow$ 39%), while hierarchical recursion contributed only a small additional gain (35.7% $\rightarrow$ 39.0%). However, this analysis did not isolate the gradient method as a variable.

2.3 The Confound

Table 1 illustrates the problem. HRM occupies the top-left cell of the 2×2 matrix (dual network, 1-step gradient). TRM occupies the bottom-right cell (flat network, full BPTT). The two papers are separated by *both* architecture and gradient method, making any direct comparison between them confounded.

Since TRM changed two variables at once, its 32-point improvement over HRM cannot be attributed to either factor without additional experiments. Our work fills the top-right cell, flat architecture with 1-step gradient, to isolate the contribution of the gradient method alone.

3 Method

We conduct a controlled experiment that varies only the gradient method while holding all other variables constant.

3.1 Architecture

We use TRM’s exact architecture [Jolicoeur-Martineau(2025)]: a single 2-layer Transformer network with RMSNorm, SwiGLU activations, rotary position embeddings, and no bias terms. The model maintains two latent features: z (reasoning state) and y (current answer embedding). Each supervision segment consists of $n = 6$ latent reasoning steps ($z \leftarrow \text{net}(x, y, z)$) followed by one answer refinement step ($y \leftarrow \text{net}(y, z)$). Training uses $T = 3$ supervision segments with deep supervision and Adaptive Computation Time (ACT). We use Exponential Moving Average (EMA) of weights with rate 0.999.

3.2 Gradient Method Manipulation

The only variable we change is where the `torch.no_grad()` boundary falls in the forward pass. In the original TRM implementation, the first $T - 1 = 2$ supervision segments (each containing $n = 6$ latent steps and one y update) execute under `torch.no_grad()`. The final segment executes with full gradient tracking, meaning gradients flow through 7 transformer layer passes (6 latent + 1 answer update).

In our modified version (TRM_{1-step}), we move *all* operations inside `torch.no_grad()` except the final answer refinement step. Specifically, all T supervision segments, including all latent reasoning steps and all intermediate answer updates, execute without gradient tracking. The gradient flows through *only* the final $y \leftarrow \text{net}(y, z)$ update, a single transformer layer pass.

This changes the gradient depth from 7 differentiable operations to 1, reducing memory from $O(T)$ to $O(1)$. The actual forward computation (the sequence of hidden state updates) is identical in both versions.

3.3 Training Setup

Both models use identical configurations, summarized in Table 2. The AdamW optimizer [Loshchilov and Hutter(2019)] replaces the original adam-atan2 [ada(2024)] for hardware compatibility; both are adaptive gradient methods and this substitution does not interact with the gradient depth variable under study.

4 Experiments

We train both model variants on Sudoku-Extreme [Sapient Intelligence(2025)], a dataset of 9×9 Sudoku puzzles ranging in difficulty. We use 500 training puzzles with 500 augmentations (random digit permutation, row/column band shuffling, and transposition), producing 1,000 effective training examples. Evaluation is performed on 2,000 held-out test puzzles, reporting both token-level accuracy (fraction of individual cells correct) and exact accuracy (fraction of fully correct puzzles).

```

1 # TRM original (full BPTT, 7 grad steps):
2 with torch.no_grad():
3     for _H_step in range(H_cycles-1):
4         for _L_step in range(L_cycles):
5             z_L = L_level(z_L, z_H + x, **seq_info)
6             z_H = L_level(z_H, z_L, **seq_info)
7 for _L_step in range(L_cycles):
8     z_L = L_level(z_L, z_H + x, **seq_info) # GRAD
9     z_H = L_level(z_H, z_L, **seq_info) # GRAD
10
11 # TRM_1-step (O(1) memory, 1 grad step):
12 with torch.no_grad():
13     for _H_step in range(H_cycles-1):
14         for _L_step in range(L_cycles):
15             z_L = L_level(z_L, z_H + x, **seq_info)
16             z_H = L_level(z_H, z_L, **seq_info)
17     for _L_step in range(L_cycles):
18         z_L = L_level(z_L, z_H + x, **seq_info) # no grad
19         z_H = L_level(z_H, z_L, **seq_info) # no grad
20     z_H = L_level(z_H, z_L, **seq_info) # GRAD <- 1 step
    
```

Figure 1: The gradient patch. Top: original TRM with full BPTT through the final supervision segment (7 gradient steps). Bottom: our 1-step version with gradient flow only through the final y update (1 step). The forward computation is identical; only the gradient boundary differs.

Table 2: Training configuration for both models. All parameters identical except gradient method.

Parameter	Value
Model	Flat TRM (single network)
Hidden size	384
Layers (L_{layers})	2
Attention heads	8
H-cycles	3
L-cycles	6
ACT halt steps	4
Batch size (global)	512 (256 per GPU)
Epochs	10,000
Training examples	500 + 500 augmentations = 1,000
Optimizer	AdamW ($\eta = 10^{-4}$, $\beta_{1,2} = 0.9, 0.95$)
Loss	stablemax cross-entropy
EMA rate	0.999
Precision	float32
Hardware	2× NVIDIA Tesla T4 (16GB)

4.1 Results

Table 3 presents our main results using the 2×2 factorial framework. The full BPTT model achieves 18.9% exact accuracy, while the 1-step gradient model achieves only 2.2%, an $8.6\times$ reduction. Token-level accuracy drops from 71.6% to 65.9%, a decrease of 5.7 percentage points (8.0% relative). This pattern suggests that the 1-step gradient approximation is sufficient for learning local patterns (individual cell predictions) but fundamentally prevents the model from learning the global constraints needed for complete puzzle solutions.

4.2 Training Dynamics

Figure 2 shows the training dynamics over 10,000 epochs. Full BPTT achieves consistently lower loss and higher exact accuracy throughout training. The gap between the two methods widens over time: at step 2,000, the 1-step and full BPTT models have comparable exact accuracy (both near 0%); by step 9,000, full BPTT reaches $\sim 15\%$ while 1-step

Table 3: Results of our controlled experiment. Both models use the same flat TRM architecture and identical training configuration. The only difference is the gradient method. Each cell shows *token accuracy % / exact accuracy %* (exact in bold). The 2×2 matrix now has three of four cells filled; the left column (HRM architecture) is left for future work.

	Dual L/H	Single flat (ours)
1-step ($O(1)$)	[Future work]	65.9% / 2.2%
Full BPTT ($O(T)$)	[Future work]	71.6% / 18.9%

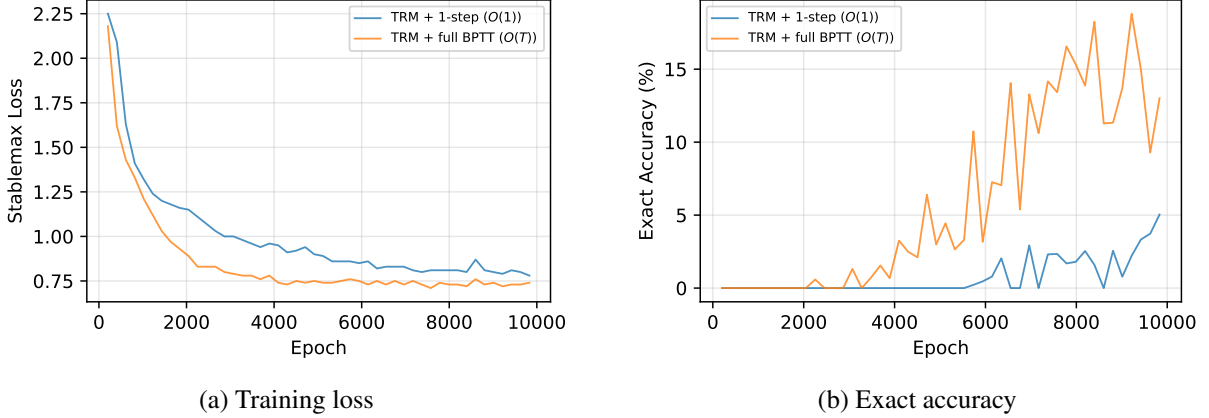


Figure 2: Training dynamics for both gradient methods. (a) Stablemax cross-entropy loss decreases faster and further under full BPTT. (b) Exact accuracy on the training set grows steadily under full BPTT, reaching 13–15% by the end of training (final test accuracy: 18.9%), while the 1-step model remains near zero with only marginal improvement. The 1-step model learns token-level patterns (loss decreases, token accuracy reaches 65.9%) but fails to assemble them into complete solutions.

remains below 5%. This divergence confirms that the gradient bottleneck is not merely slowing convergence; it is preventing the model from ever learning the full reasoning chain.

4.3 Training Efficiency

The 1-step model trains faster per step (1.63s vs 2.73s, $1.67\times$ speedup) due to reduced backward pass computation. However, the per-step training cost is dwarfed by the accuracy gap: full BPTT achieves $8.6\times$ higher exact accuracy for only $1.67\times$ more compute per step. The $O(1)$ memory advantage of the 1-step approximation becomes relevant only when recursion depth T is large enough that full BPTT exceeds GPU memory, a regime we do not reach in our experiments ($T = 7$).

5 Discussion

Our results establish that gradient quality is also a dominant factor in recursive reasoning performance, not architecture alone. The flat TRM architecture *is* capable of learning to solve Sudoku puzzles, reaching 18.9% exact accuracy, when given proper gradient signals. The same architecture crippled by the 1-step approximation achieves only 2.2%, barely above the level achievable by learning token-level patterns alone.

This finding has several implications. First, it suggests that the community’s attribution of TRM’s success to only architecture simplification [ARC Prize Foundation(2025)] was premature; the results were confounded by the simultaneous change in gradient method. Second, the $8.6\times$ collapse in exact accuracy occurs despite token-level accuracy dropping only modestly (71.6% $\rightarrow$ 65.9%). The 1-step approximation preserves the model’s ability to predict individual cells correctly, but cripples its ability to coordinate them into globally consistent solutions. Gradient quality, not architectural capacity, is what enables the model to satisfy long-range constraints. Third, it establishes a clean experimental framework for future work: the 2×2 factorial design (Table 3) should be the default when comparing recursive architectures.

Our work has several limitations. We evaluate on a single task (Sudoku-Extreme) with a moderate model size (384 hidden dimensions, 2 layers). The relative importance of gradient quality vs. architecture may differ at larger scales or on tasks requiring different reasoning strategies (e.g., maze pathfinding, abstract reasoning on ARC-AGI). Additionally, our full BPTT model achieves 18.9% exact accuracy, substantially below TRM’s reported 87.4%. This is expected given our reduced training budget (10,000 vs. 60,000 epochs) and model size (384 vs. 512 hidden dimensions, 2 vs. 4 layers). Scaling to match TRM’s full configuration would likely close this gap, but is left for future work.

The most natural extension of this work is completing the 2×2 matrix: evaluating HRM’s dual-network hierarchical architecture with full BPTT. This would answer whether the brain-inspired hierarchy provides any benefit once the gradient bottleneck is removed, or whether architecture is truly irrelevant for recursive reasoning performance. We leave this experiment to future work.

6 Conclusion

We have demonstrated that architecture alone is not the primary factor explaining TRM’s performance advantage over HRM; the gradient method also played an important role. By running a controlled experiment that isolates the gradient method, we show that the 1-step gradient approximation reduces exact accuracy from 18.9% to 2.2%, an $8.6\times$ gap, while the flat architecture remains identical. This resolves a confound that has gone unaddressed in the recursive reasoning literature and establishes a clean experimental paradigm for future comparisons. The 2×2 factorial matrix provides a roadmap: our work fills the top-right and bottom-right cells (flat architecture at both gradient methods); the remaining task is evaluating the left column (hierarchical architecture at both gradient methods).

References

- [Wang et al.(2025)Wang, Li, Sun, Chen, Liu, Wu, Lu, Song, and Yadkori] Guan Wang, Jin Li, Yuhao Sun, Xing Chen, Changling Liu, Yue Wu, Meng Lu, Sen Song, and Yasin Abbasi Yadkori. Hierarchical reasoning model. *arXiv preprint arXiv:2506.21734*, 2025.
- [Jolicoeur-Martineau(2025)] Alexia Jolicoeur-Martineau. Less is more: Recursive reasoning with tiny networks. *arXiv preprint arXiv:2510.04871*, 2025.
- [Bai et al.(2019)Bai, Kolter, and Koltun] Shaojie Bai, J Zico Kolter, and Vladlen Koltun. Deep equilibrium models. In *Advances in Neural Information Processing Systems*, 2019.
- [ARC Prize Foundation(2025)] ARC Prize Foundation. Hrm ablation analysis on arc-agi. <https://arcprize.org/blog/hrm-ablation>, 2025.
- [Loshchilov and Hutter(2019)] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019.
- [ada(2024)] Adam-atan2 optimizer. <https://github.com/kyegomez/Adam-atan2>, 2024.
- [Sapient Intelligence(2025)] Sapient Intelligence. Sudoku-extreme dataset. <https://huggingface.co/datasets/sapientinc/sudoku-extreme>, 2025.